

KasSee Web

User Guide

Watch-only companion wallet for KasSigner. Pure Rust in the browser, zero install; your keys never leave the device.

Release `v1.0.7`

Date `August 2026`

Licence `GPL-3.0`

Repository `github.com/InKasWeRust/KasSigner`

Contact `kassigner@proton.me`

KasSee is a watch-only wallet. It never sees your private keys. It runs in your browser, an untrusted environment. Always verify transaction details on the KasSigner device screen before signing. The device is the only trusted display in the system.

Don't trust, verify. Sign offline. Broadcast sovereign.

Contents

1. What is KasSee Web?
2. Getting Started
3. Importing Your kpub
4. Dashboard
5. Sending KAS
6. Receiving KAS
7. Broadcasting Signed Transactions
8. Address Verification
9. UTXO Management
10. Node Configuration
11. Caddy TLS Proxy (Advanced)
12. Multisig (45')
13. Covenants++ and Stealth Payments
14. Token Display
15. Security Model
16. Troubleshooting
17. Donation

1. What is KasSee Web?

KasSee Web is a browser-based, watch-only companion wallet for the KasSigner air-gapped signing device. It runs entirely in your browser: no installation, no extensions, no app store. Built from Rust compiled to WebAssembly, it does what an online wallet does, checks balances, builds transactions, shows QR codes, broadcasts signed transactions, and it never touches your private keys.

KasSee connects to the Kasper network through public nodes (or your own) over Borsh wRPC. It imports your extended public key (kpub) from KasSigner, derives your addresses, and manages your UTXOs. When you send, it builds an unsigned transaction and shows it as an animated QR. KasSigner scans it, signs offline, and shows the signed result as another QR. KasSee scans that back and broadcasts.

KasSee is also the project's test bench: new device features, covenant designs and ideas are tried here first. It is not the product; KasSigner speaks PSKB and QR and works with any watch-only wallet that does the same.

Key properties

- **Zero install:** open kassigner.org and start
- **Rust/WASM:** all cryptography in Rust, no JavaScript crypto
- **Air-gapped:** private keys never leave your KasSigner
- **Zero config:** auto-connects to public Kasper nodes via resolver
- **Open source:** GPL-3.0, built from the source in the repository; the release build compile-verifies it

2. Getting Started

Requirements

- A modern browser (Safari, Chrome, Brave, Firefox)
- A KasSigner device with a loaded seed and exported kpub
- Internet connection for the node
- Camera access for scanning QR codes, which requires HTTPS

Access

Open kassigner.org. On mobile, add it to your home screen; the page works as a Progressive Web App.

3. Importing Your kpub

Your kpub is the watch-only key exported from KasSigner. It lets KasSee derive your receive and change addresses and watch the balance; it cannot sign or spend.

Option A: Scan QR

- On KasSigner: Seeds, Export, Watch-Only, **kpub as QR**
- In KasSee, tap Scan kpub QR and point the camera at the device
- KasSee imports the kpub and opens the dashboard

Option B: Paste

- Copy the kpub string (starts with kpub) and paste it into the welcome screen, tap Import kpub

KasSee derives 20 receive and 10 change addresses. The kpub is stored in the browser's localStorage; it persists across sessions and is cleared with Reset Wallet.

The device also exports a **kpub Multisig QR**. That is the 45' key for multisig wallets, not the watch-only key. The two look identical; scan the one the label says.

4. Dashboard

After import, KasSee connects to a node, fetches the UTXOs of all your addresses and shows the total balance. The dashboard offers:

- **Send KAS**: build an unsigned transaction
- **Receive**: your next unused receive address as QR
- **Broadcast TX**: broadcast a signed transaction from KasSigner
- **Addresses**: view and verify every derived address, with funded/used badges and explorer links
- **UTXOs**: view, select and consolidate
- **Multisig**: 45' multisig wallets (chapter 12)
- **Covenants++** and **Stealth**: chapter 13
- **Tokens**: KRC-20, KRC-721, KNS (read-only)
- **Node Settings** and **Refresh Balance**

The status indicator in the header shows red (offline), yellow (connecting), green (connected). If a node fails, KasSee resolves another; nodes that stop answering are remembered and skipped for the session.

5. Sending KAS

Building the transaction

- Tap Send KAS, enter the destination kaspa: address and the amount (or tap Max)
- Choose a fee level: Low (2,500), Normal (5,000) or Priority (10,000 sompi and up)
- Optionally tap **Select UTXOs manually** to choose the inputs, up to 32
- Tap Create Transaction: KasSee builds an unsigned KSPT and shows it as an animated QR

Signing on KasSigner

- Point the KasSigner camera at the QR
- The device assembles the frames, shows every output for review, the lock time if the transaction carries one, and asks for confirmation
- Confirm on device: KasSigner signs offline and shows the signed QR, single or multi-frame

Broadcasting

- In KasSee, Broadcast TX, Scan Signed QR, point the camera at the device
- Teal dots light as each frame is captured; when all are in, KasSee broadcasts
- You can also paste the signed hex if the camera is not available

6. Receiving KAS

Tap Receive to show your next unused receive address as a QR. KasSee skips funded and previously used addresses so you do not reuse one. Compare the address with the one your KasSigner shows for the same index before sharing it.

7. Broadcasting Signed Transactions

The broadcast screen scans the signed QR from KasSigner (single-sig KSPT or a fully signed multisig bundle), assembles the frames and submits to the connected node. A partially signed multisig bundle can also be scanned here to relay it on to the next cosigner (chapter 12).

8. Address Verification

Tap Addresses to see all receive and change addresses with their derivation paths and indices. Each address shows:

- Orange **funded** badge: currently holds UTXOs
- Red **used** badge: was funded, now empty; do not reuse
- Explorer link: open the address on the Kaspas block explorer

Tap an address for the verification screen: full address in large text, its QR, the exact derivation path (e.g. m/44'/111111'/0'/0/13) and an explorer link. Compare with the address on your KasSigner. Long-press to copy.

Address History (advanced)

In Settings you can enable used-address detection through a self-hosted kaspas-rest-server (archival node with DB indexer) by giving its REST URL. Public APIs do not support this.

9. UTXO Management

Tap UTXOs to see all unspent outputs, largest first, with amount and outpoint.

Consolidate Selected

Each UTXO has a checkbox. Select up to 32, the signer's input limit; with two or more selected, **Consolidate N Selected** builds a transaction merging them into one output to your first receive address. Repeat in batches.

Consolidate All

With nothing selected, **Consolidate All** merges the 32 largest UTXOs into one output. Run it again until the count is where you want it.

Manual selection when sending

On the Send screen, **Select UTXOs manually** expands the selector. Pick up to 32 inputs; Max recalculates the sendable amount from your selection. Selecting more than 32 is refused with a message.

Why consolidate: many small UTXOs left by many payments push a spend past the signer's 32-input limit, and below the network's storage-mass threshold they can become impossible to spend at all. Merge them while they are still spendable.

10. Node Configuration

By default KasSee connects to public Kaspas nodes through a resolver. To use your own node, tap Node Settings and enter its wRPC Borsh URL; the node needs Borsh wRPC enabled:

```
--rpclisten-borsh=0.0.0.0:17110
```

URL format

- From localhost (HTTP page): ws://127.0.0.1:17110
- From an HTTPS page (kassigner.org): wss:// only, through a TLS proxy
- From a Caddy proxy: wss://192.168.x.x:8444 (chapter 11)

The custom URL is saved in localStorage; Reset to Public Nodes clears it. KasSee keeps one WebSocket per node for the session and re-resolves on timeout rather than hammering a dead endpoint.

11. Caddy TLS Proxy (Advanced)

To use KasSee on your phone with camera access AND your own local Kasper node, you need HTTPS. Caddy provides a local CA and TLS for your LAN.

Setup

- Install Caddy (brew install caddy)
- Create a Caddyfile with two ports: one serving KasSee (8443), one proxying the node's Borsh wRPC (8444)
- Global options must include **protocols h1**: the node's wRPC endpoint speaks HTTP/1 only, so the WebSocket upgrade fails through HTTP/2. Use **tls internal** and run **caddy trust** for the local CA
- Run caddy with the config, trust the root certificate on your phone (AirDrop root.crt, install the profile)
- Open `https://<your-lan-ip>:8443` on the phone and set the node URL to `wss://<your-lan-ip>:8444` in Node Settings

Your phone now reaches KasSee over HTTPS (camera works) and your node over WSS. Everything stays on your network.

12. Multisig (45')

KasSee supports the Kasper 45' multisig scheme (KIP: Multisig Wallet Conventions for Kasper, github.com/kaspanet/kips/pull/39). The wallet is created on the KasSigner devices; KasSee watches it, builds spends, relays bundles between cosigners and broadcasts.

Load a wallet

Multisig, **Load a descriptor and one of its addresses**: scan the descriptor QR from a device (or paste it) and scan or paste one of the wallet's addresses. The address identifies your cosigner branch, so KasSee knows which of the N address families is yours. Legacy 44' multi_hd descriptors load the same way.

Watch, receive, send

- Wallet view with balance, addresses and UTXOs across the wallet
- Receive: the next unused multisig address
- Send: pick a destination and amount, or the UTXOs to spend across several addresses; KasSee builds a PSKB with the derivation hints each device needs

Signing flow

- Show the unsigned bundle QR; Device A scans, reviews, signs and shows a partially signed bundle (PARTIAL 1/2)
- Device B scans Device A's QR, or KasSee relays it: scan the partial bundle in Broadcast and show it again for the next device
- The last device shows FULLY SIGNED; scan it in KasSee and broadcast

Back up **seed and descriptor**. A seed alone cannot find or spend multisig funds. On the device, a change output that the descriptor contradicts is refused; one it cannot verify is shown as unverified for you to check.

13. Covenants++ and Stealth Payments

KasSee carries the covenant builders (piggy bank, time-locked savings, dead man's switch, allowance, spending limit, Merkle whitelist, direct channel, oracle, PayJoin, commit-reveal, private swap, ZK price oracle) and stealth payments with dual-key addresses. Every one signs on the device the same way: the device shows every input and output, the payload hash and the covenant id, and verifies the redeem script against the address being spent. Full walkthroughs are in the **Covenants and Stealth Guide**.

14. Token Display

Tokens shows KRC-20 balances, KRC-721 NFTs and KNS domain names associated with your addresses. Read-only; KasSee does not transfer tokens.

15. Security Model

- **Private keys** never leave the KasSigner device. KasSee only has extended public keys
- **Watch-only**: KasSee sees balances and builds unsigned transactions; it cannot sign or spend
- **No server**: all logic runs in your browser; no backend, no database, no accounts, no analytics
- **Local storage**: kpub, descriptor and node URL in browser localStorage only. Clear anytime with Reset Wallet
- **HTTPS**: kassigner.org serves over HTTPS; the camera needs a secure context
- **Own node**: for privacy, connect to your own Kaspa node; a public node sees which addresses belong together, the balance, and your IP

KasSee is **not a security boundary**. A phishing clone can show one address and put another in the QR; browser malware can rewrite a transaction in memory. The final check is always on the KasSigner screen, which shows what is in the transaction data, not what the browser claims.

16. Troubleshooting

Camera not working

The camera needs HTTPS. Use kassigner.org or the Caddy proxy; http://localhost is the one exception.

Balance shows nothing

Node connection failed. Tap Refresh Balance. With a custom node, check the URL and that Borsh wRPC is enabled. Public nodes that reject or time out are skipped automatically.

Transaction broadcast fails

Check the fee, that every frame of the signed QR was captured, and, for a covenant, that the covenant address is funded.

Addresses don't match KasSigner

Reset Wallet and re-import the kpub. Compare the same index and chain (receive or change).

ws:// connection blocked

HTTPS pages cannot open ws://. Use wss:// through Caddy, or run KasSee from http://localhost on the same machine as the node.

Phone can't reach the local server

Same WiFi network, and use the computer's LAN IP, not localhost.

QR scanning is slow

Hold steady, good light, screen at full brightness, a slight tilt of 20 to 45 degrees against reflections. Frames cycle automatically; use the pause/play control to hold one, and the speed slider to slow the animation for the M5Stack decoder.

17. Donation

Tapping the KasSigner logo in the header opens a donation card with the project's Kaspa address (kassigner.kas) as a QR. If a wallet is loaded, tapping the logo pre-fills the Send screen with the donation address instead.